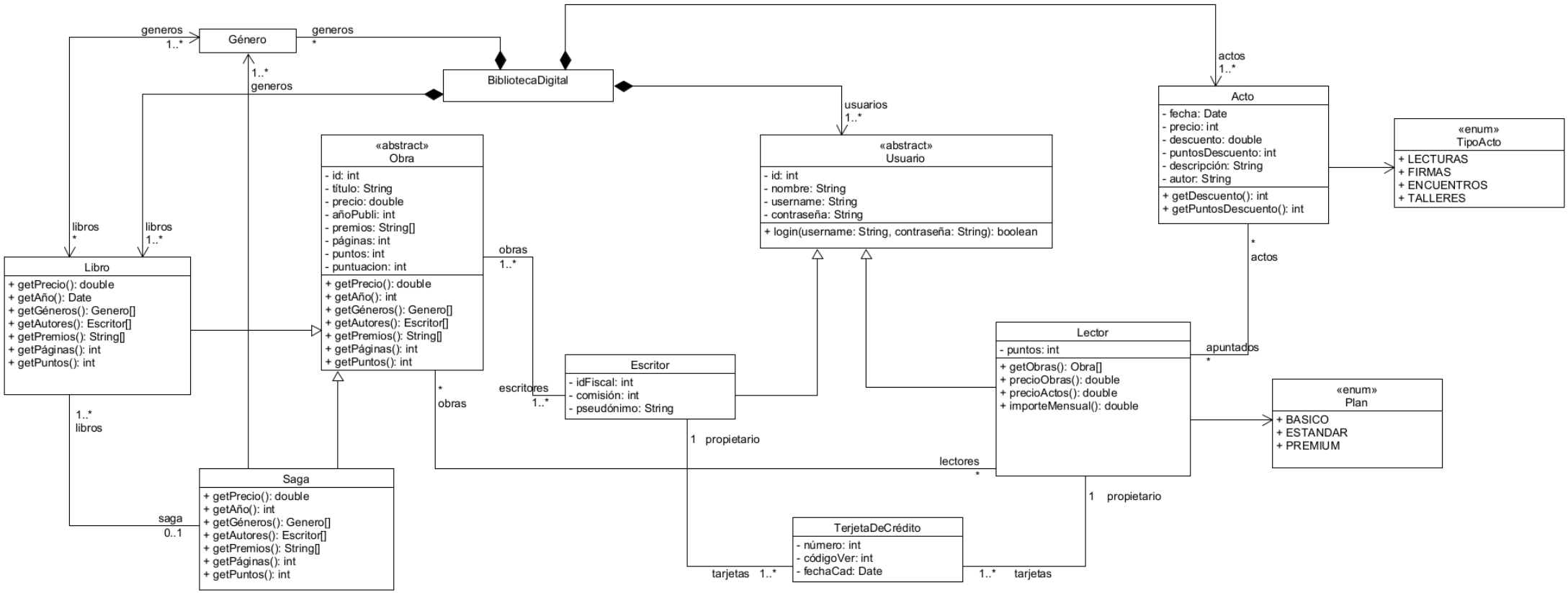
Ejercicio 2:

a,b)



En este diagrama se ve un diseño de una biblioteca digital en la que se pueden comprar libros individuales, sagas completas y acceder a diferentes actos relacionados con los escritores.

Se ha partido de una clase abstracta llamada Obra que incluye los atributos comunes de Libro y de Saga, a los que se les ha incluido los getters solicitados en el enunciado. También se ha creado una clase Género para asegurar el acceso con eficacia a todos los libros de un mismo género. Sin embargo, no se solicitaba el acceso eficiente a las sagas de un mismo Género por lo que se ha mantenido la navegación de Género a Libro y Saga por separado, siendo la de Saga unidireccional.

Respecto a los Usuarios, se han creado dos clases, Escritor y Lector. También cabe destacar las dos enumeraciones que hemos incluido, Plan, con los distintos planes de pago de los lectores, y Acto, con los posibles actos a los que se pueden optar con el sistema de puntos.

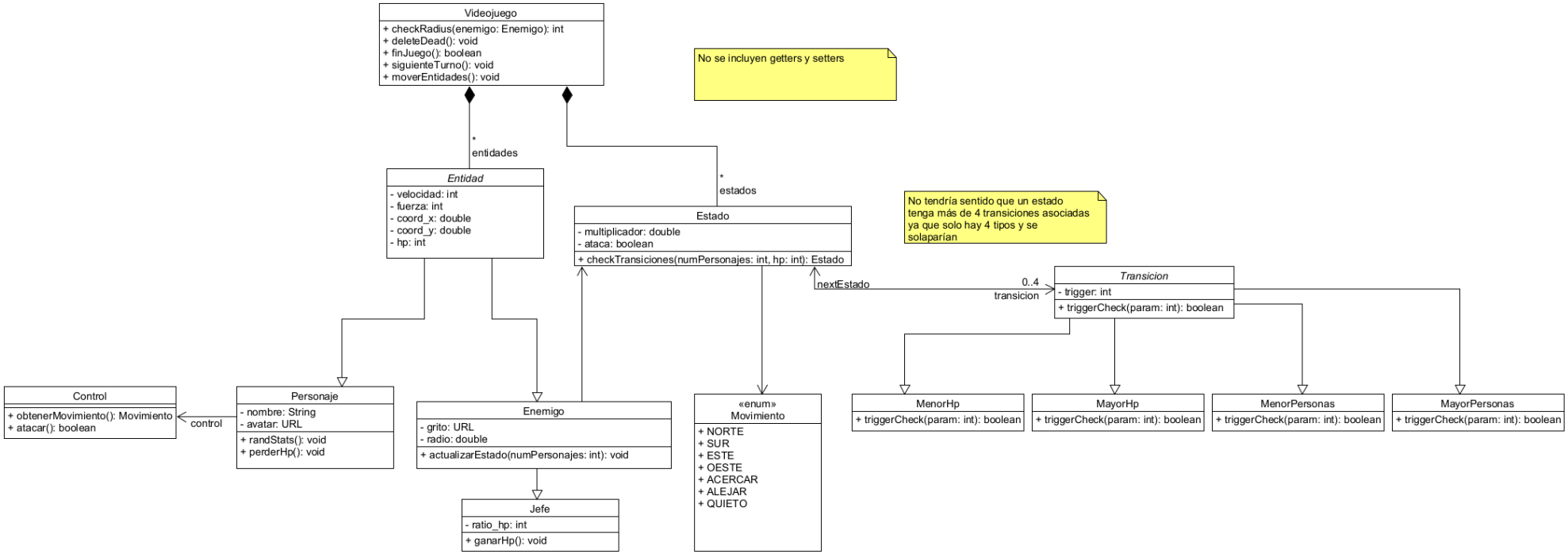
c)

Se han creado las funciones necesarias en pseudocódigo (todas pertenecientes a la clase Lector):

```
importeMensual()
  total = 0

  total += lector.precioObras()
  total += lector.precioActos()

  if this.plan == Plan.ESTANDAR then
    total /= 2
    total += 9.99
  else if this.plan == Plan.PREMIUM then
    total = 19.99

  return total
```

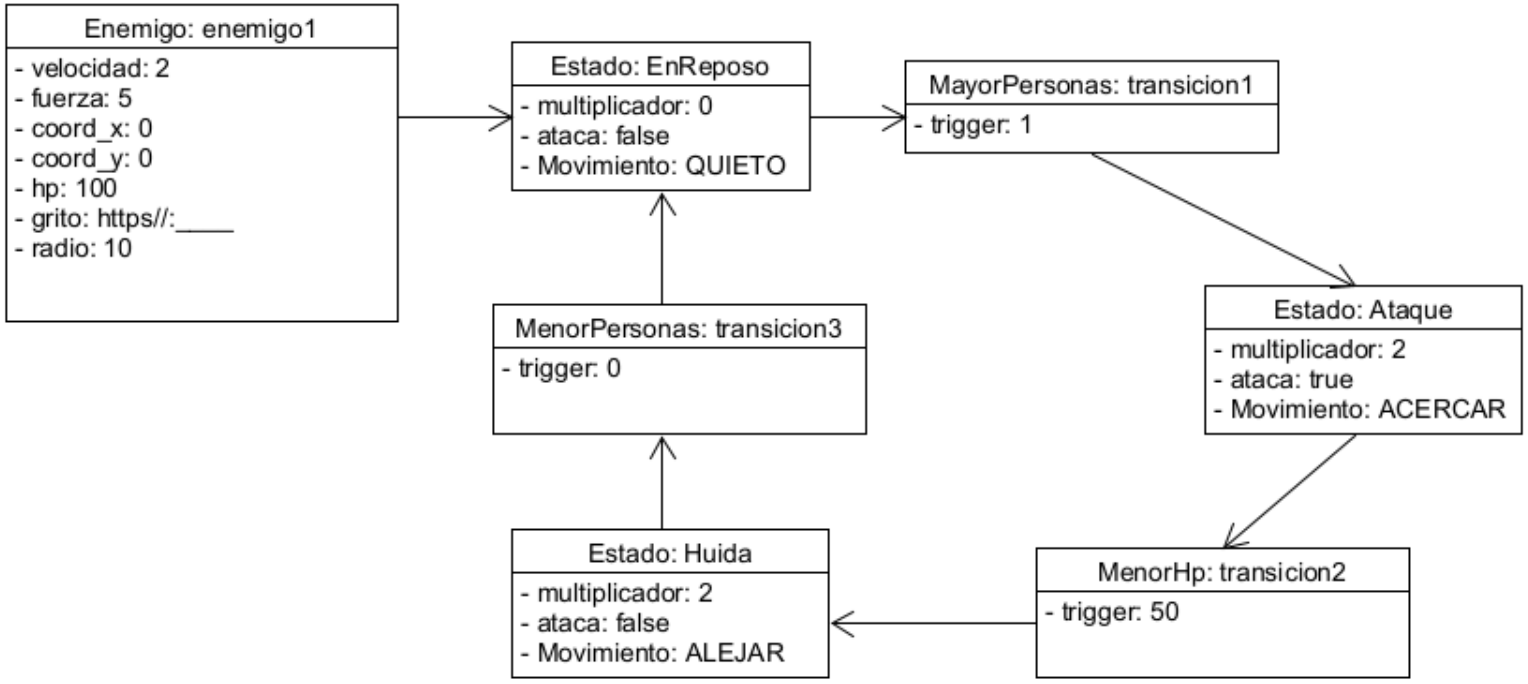
```
precioObras()
    total = 0
    for obra in this.obras:
        total += obra.getPrecio()
    return total

precioActos()
    total = 0
    for acto in this.actos:
        total += acto.getPrecio()
        if(acto.getPuntosDescuento >= this.puntos)
            // se asume que el descuento nunca será mayor
            que el precio del acto
            total -= acto.getDescuento()
    return total
```

Ejercicio 3: a)



El diagrama de clases sigue lo especificado en el enunciado, se ha decidido que los enemigos contengan una clase Estado que a su vez apunta a entre 0 y 4 transiciones, y estas apuntan a otro Estado. Cada ciclo del videojuego está marcado por el método siguienteTurno() de la clase videojuego, que realiza todas las acciones en cada turno (modificar los puntos de vida de las entidades, moverlas según lo que

indique Control en caso de los personajes y Estado en caso de los enemigos y actualizar los estados de los enemigos).

Para actualizar los estados de los enemigos, pueden llamar a un método actualizarEstado() que recibe el número de personajes en el radio del enemigo (calculado con la función checkRadius() de Videojuego). Este a su vez llama al método checkTransiciones() de su Estado, que recibe tanto el número de personajes como la vida del enemigo. Después este comprueba si se cumplen las condiciones de alguna de las transiciones (con triggerCheck()) y devuelve el estado al que apunta en caso de que se cumpla la condición de dicha transición.

b)